



SYNTROPY

Don't manage time. Execute it.

HACKATHON MVP IMPLEMENTATION PLAN

Created for: VibeCode 2.0 Hackathon

Platform: Next.js + Google Gemini 1.5 Pro + GCP

Generated: 2026-06-30

[Award] Syntropy Phase 4 Real-Time Workspace Integrations R

This report outlines the technical architecture to transition Syntropy from a high-fidelity prototype to a fully integrated platform. It permits users to manage their own tasks in real-time and links their actions directly to live **Gmail**, **Google Docs**, and **Google Drive** instances.

1. Real-Time Data Flow & Firestore Sync

To allow users to input their own tasks and sync them instantly across sessions, we will replace the static `initialTasks` mock data array with a live **Google Cloud Firestore** listener.

```
graph LR
    User[User Input Form] -->|Add/Edit Task| Client[Next.js Client]
    Client -->|Write| Firestore[(Cloud Firestore)]
    Firestore -->|Real-time update onSnapshot| Client
    Client -->|Sync state| Timeline[Timeline View]
```

Firestore Schema Details:

- `users/{userId}`: User profile doc containing metadata, burnout levels, and integration parameters.
- `users/{userId}/tasks/{taskId}`: Document containing:
 - `title`, `description`, `time`, `category` (Work, Academic, Client)
 - `risk` (low, high, critical)
 - `status` (unprepared, preparing, prepared, at-risk, negotiating, negotiated)
 - `prepContent` (AI-generated draft text)
 - `approved` (boolean check flag)

2. Google Workspace OAuth & API Integration

When a user logs in via Google Sign-In, we will request extended OAuth permission scopes to let the application perform actions on their behalf.

Required Scopes:

1. **Gmail** (`.../auth/gmail.compose`): Required to create draft emails in the user's Gmail box.
2. **Google Docs** (`.../auth/documents`): Required to create and modify outline documents.
3. **Google Drive** (`.../auth/drive.file`): Required to organize and save pre-work files.

```
sequenceDiagram
    participant User as User Browser
    participant App as Syntropy Client
    participant Auth as Firebase Google Auth
    participant GoogleAPI as Google REST APIs (Gmail/Docs)

    User->>App: Click "Approve & Sync"
    App->>Auth: Retrieve Google Access Token
    App->>GoogleAPI: POST /v1/documents (Create outline Doc)
    GoogleAPI-->>App: Return Doc URL
    App->>User: Display live link: "Open Google Doc"
```

Authentication Token Handling:

During the `loginWithGoogle` popup flow, the `GoogleAuthProvider` credential exposes an `accessToken`. We will store this token in the client state (and session) to authorize REST API calls directly to:

- **Google Docs API:** `POST https://docs.googleapis.com/v1/documents`
- **Gmail API:** `POST https://gmail.googleapis.com/v1/users/me/drafts`

3. UI/UX: Task Input Widgets & Live Feedback

We will design a modern glassmorphic task insertion drawer:

- **Interactive Form:** Inputs for Title, Deadline, Description, Category, and context threads.
- **Autonomous AI Ingest:** Upon saving a task to Firestore, a background hook triggers the AI Anticipator. The system starts creating pre-work outlines or analyzing schedule risks immediately.

4. Implementation Targets & Milestones

Phase 1: Real-Time Firestore Sync

- Configure Firestore client SDK.
- Replace `useState(initialTasks)` in `dashboard/page.js` with `onSnapshot` listeners bound to the Firebase Auth user's collections.
- Build the "Add Task" modal form.

Phase 2: Gmail Composition Integration

- Update Google Auth configuration to request Gmail compose scopes.
- Build a REST API connector client to generate real draft messages inside the user's Gmail workspace.

Phase 3: Google Docs Assembly Integration

-
- Build a REST API connector client to write to the Google Docs endpoint, generating documents under the user's account.